

Assignment 1: Part-of-Speech Tagging using Hidden Markov Models (HMMs)

Course: Natural Language Processing S2-25_AIMLCZG530

Dataset: NLTK Brown Corpus

Training Category: news

Testing Category: fiction

Team Details

Group 108

Name	BITS ID
PRASHANT .	2025ab05113
ARTHIKA G .	2025ab05180
ASWATHY H .	2025ab05203
SRINEVEDA R S .	2025ab05206
SUKANYA YADAV .	2025aa05630

Setup

```
In [1]: import platform
import socket
import sys

print("=" * 60)
print("Environment Information")
print("=" * 60)

print(f"Machine Name : {socket.gethostname()}")
print(f"Operating System : {platform.system()} {platform.release()}")
print(f"Machine Type : {platform.machine()}")
print(f"Processor : {platform.processor()}")
print(f"Python Version : {sys.version.split()[0]}")
```

```
=====
Environment Information
=====
Machine Name : 2025ab05113
Operating System : Linux 5.14.0-503.31.1.el9_5.x86_64
Machine Type : x86_64
Processor : x86_64
Python Version : 3.9.21
```

```
In [2]: import math
import nltk
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from collections import Counter, defaultdict
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

nltk.download("brown", quiet=True)
nltk.download("universal_tagset", quiet=True)
nltk.download("averaged_perceptron_tagger", quiet=True)
nltk.download("averaged_perceptron_tagger_eng", quiet=True)

from nltk.corpus import brown
```

Part 1: Data Preprocessing & Exploratory Data Analysis (EDA)

1(a) Data Loading

```
In [3]: train_sentences_raw = brown.tagged_sents(categories="news", tagset="universal")
test_sentences_raw = brown.tagged_sents(categories="fiction", tagset="universal")

print("Training category: news")
print("Testing category: fiction")
print("Number of training sentences:", len(train_sentences_raw))
print("Number of testing sentences:", len(test_sentences_raw))
print("\nSample training sentence:")
print(train_sentences_raw[0])
print("\nSample testing sentence:")
print(test_sentences_raw[0])
```

Training category: news
 Testing category: fiction
 Number of training sentences: 4623
 Number of testing sentences: 4249

Sample training sentence:
 [('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'), ('investigation', 'NOUN'), ('of', 'ADP'), ('Atlanta's', 'NOUN'), ('recent', 'ADJ'), ('primary', 'NOUN'), ('election', 'NOUN'), ('produced', 'VERB'), ('', '.'), ('no', 'DET'), ('evidence', 'NOUN'), ('', '.'), ('that', 'ADP'), ('any', 'DET'), ('irregularities', 'NOUN'), ('took', 'VERB'), ('place', 'NOUN'), ('.', '.')]]

Sample testing sentence:
 [('Thirty-three', 'NUM')]

1(b) Exploratory Data Analysis (EDA)

```
In [4]: train_words_original = [word for sent in train_sentences_raw for word, tag in sent]
train_tags = [tag for sent in train_sentences_raw for word, tag in sent]
test_words_original = [word for sent in test_sentences_raw for word, tag in sent]
test_tags = [tag for sent in test_sentences_raw for word, tag in sent]

eda_summary = pd.DataFrame({
    "Metric": ["Training sentences", "Testing sentences", "Training tokens", "Testing tokens",
              "Unique training words", "Unique testing words", "Unique POS tags"],
    "Value": [len(train_sentences_raw), len(test_sentences_raw), len(train_words_original),
              len(test_words_original), len(set(train_words_original)), len(set(test_words_original)),
              len(set(train_tags))]
})
eda_summary
```

Out[4]:

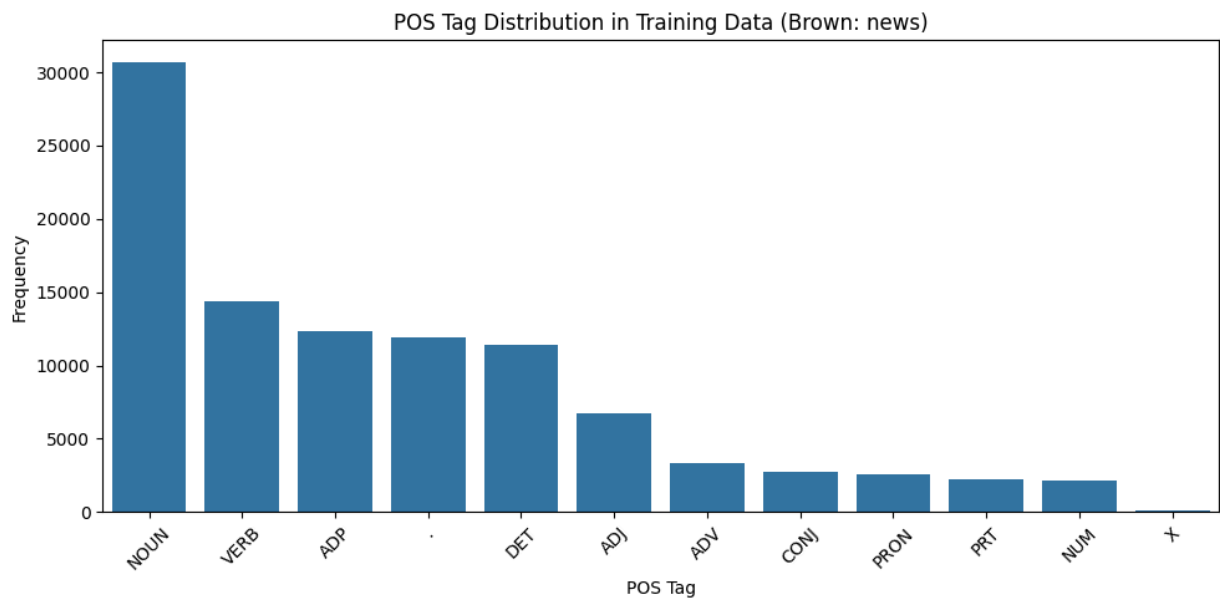
	Metric	Value
0	Training sentences	4623
1	Testing sentences	4249
2	Training tokens	100554
3	Testing tokens	68488
4	Unique training words	14394
5	Unique testing words	9302
6	Unique POS tags	12

```
In [5]: tag_distribution_df = (
    pd.DataFrame(Counter(train_tags).items(), columns=["POS Tag", "Frequency"])
    .sort_values("Frequency", ascending=False)
    .reset_index(drop=True)
)
tag_distribution_df
```

Out [5]:

	POS Tag	Frequency
0	NOUN	30654
1	VERB	14399
2	ADP	12355
3	.	11928
4	DET	11389
5	ADJ	6706
6	ADV	3349
7	CONJ	2717
8	PRON	2535
9	PRT	2264
10	NUM	2166
11	X	92

```
In [6]: plt.figure(figsize=(10, 5))
sns.barplot(data=tag_distribution_df, x="POS Tag", y="Frequency")
plt.title("POS Tag Distribution in Training Data (Brown: news)")
plt.xlabel("POS Tag")
plt.ylabel("Frequency")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



1(c) Vocabulary Handling & Unknown Words

```
In [7]: def normalize_word(word):
return word.lower()

train_vocabulary = set(normalize_word(word) for word in train_words_original)
test_vocabulary = set(normalize_word(word) for word in test_words_original)
oov_words = test_vocabulary - train_vocabulary

vocab_summary = pd.DataFrame({
    "Metric": ["Training vocabulary size", "Testing vocabulary size", "OOV words in testing data",
              "OOV percentage in testing vocabulary"],
    "Value": [len(train_vocabulary), len(test_vocabulary), len(oov_words),
              round((len(oov_words) / len(test_vocabulary)) * 100, 2)]
})
vocab_summary
```

Out [7]:

	Metric	Value
0	Training vocabulary size	13112.0
1	Testing vocabulary size	8680.0
2	OOV words in testing data	4609.0
3	OOV percentage in testing vocabulary	53.1

In [8]: `pd.DataFrame(sorted(list(oov_words))[:25], columns=["Sample OOV Words"])`

Out [8]:

	Sample OOV Words
0	\$2,500
1	'low
2	'mon
3	'nother
4	'nuff
5	'scuse
6	'tis
7	0600
8	106
9	1492
10	1939
11	2130
12	2230
13	7:00
14	7:25
15	8-balls
16	a-coming
17	abandoning
18	abbe
19	abbot
20	abdomen
21	abed
22	ablard
23	abortion
24	abruptly

1(d) Sentence Padding

In [9]:

```
START_TAG = "<START>"
END_TAG = "<END>"
UNK_TOKEN = "<UNK>"

def pad_tagged_sentence(sentence):
    return [(START_TAG, START_TAG)] + list(sentence) + [(END_TAG, END_TAG)]

padded_example = pad_tagged_sentence(train_sentences_raw[0])
print("Original sentence length:", len(train_sentences_raw[0]))
print("Padded sentence length:", len(padded_example))
print("\nPadded sentence example:")
print(padded_example[:10], "...", padded_example[-3:])
```

Original sentence length: 25
Padded sentence length: 27

Padded sentence example:

```
[('<START>', '<START>'), ('The', 'DET'), ('Fulton', 'NOUN'), ('County', 'NOUN'), ('Grand', 'ADJ'), ('Jury', 'NOUN'), ('said', 'VERB'), ('Friday', 'NOUN'), ('an', 'DET'), ('investigation', 'NOUN')] ... [('place', 'NOUN'), ('.', '.'), ('<END>', '<END>')]
```

1(e) Data Splitting

```
In [10]: split_summary = pd.DataFrame({
    "Dataset Role": ["Training", "Testing"],
    "Brown Category": ["news", "fiction"],
    "Number of Sentences": [len(train_sentences_raw), len(test_sentences_raw)],
    "Number of Tokens": [len(train_words_original), len(test_words_original)]
})
split_summary
```

```
Out[10]:
```

	Dataset Role	Brown Category	Number of Sentences	Number of Tokens
0	Training	news	4623	100554
1	Testing	fiction	4249	68488

Model Architecture

2(a) HMM States and Observations

```
In [11]: states = sorted(set(train_tags))
observations = sorted(train_vocabulary)

pd.DataFrame({
    "Component": ["Hidden states", "Observations", "Start state", "End state"],
    "Value": [f"{len(states)} POS tags", f"{len(observations)} normalized training words", START_TAG, END_TAG]
})
```

```
Out[11]:
```

	Component	Value
0	Hidden states	12 POS tags
1	Observations	13112 normalized training words
2	Start state	<START>
3	End state	<END>

2(b) Maximum Likelihood Estimation: Transition Matrix (A)

```
In [12]: transition_counts = defaultdict(Counter)
tag_context_counts = Counter()

for sentence in train_sentences_raw:
    padded_sentence = pad_tagged_sentence(sentence)
    padded_tags = [tag for word, tag in padded_sentence]
    for i in range(len(padded_tags) - 1):
        prev_tag = padded_tags[i]
        next_tag = padded_tags[i + 1]
        transition_counts[prev_tag][next_tag] += 1
        tag_context_counts[prev_tag] += 1

all_next_tags = states + [END_TAG]
all_prev_tags = [START_TAG] + states
num_next_tags = len(all_next_tags)

transition_log_probs = defaultdict(dict)
for prev_tag in all_prev_tags:
    denominator = tag_context_counts[prev_tag] + num_next_tags
    for next_tag in all_next_tags:
        numerator = transition_counts[prev_tag][next_tag] + 1
        transition_log_probs[prev_tag][next_tag] = math.log(numerator / denominator)

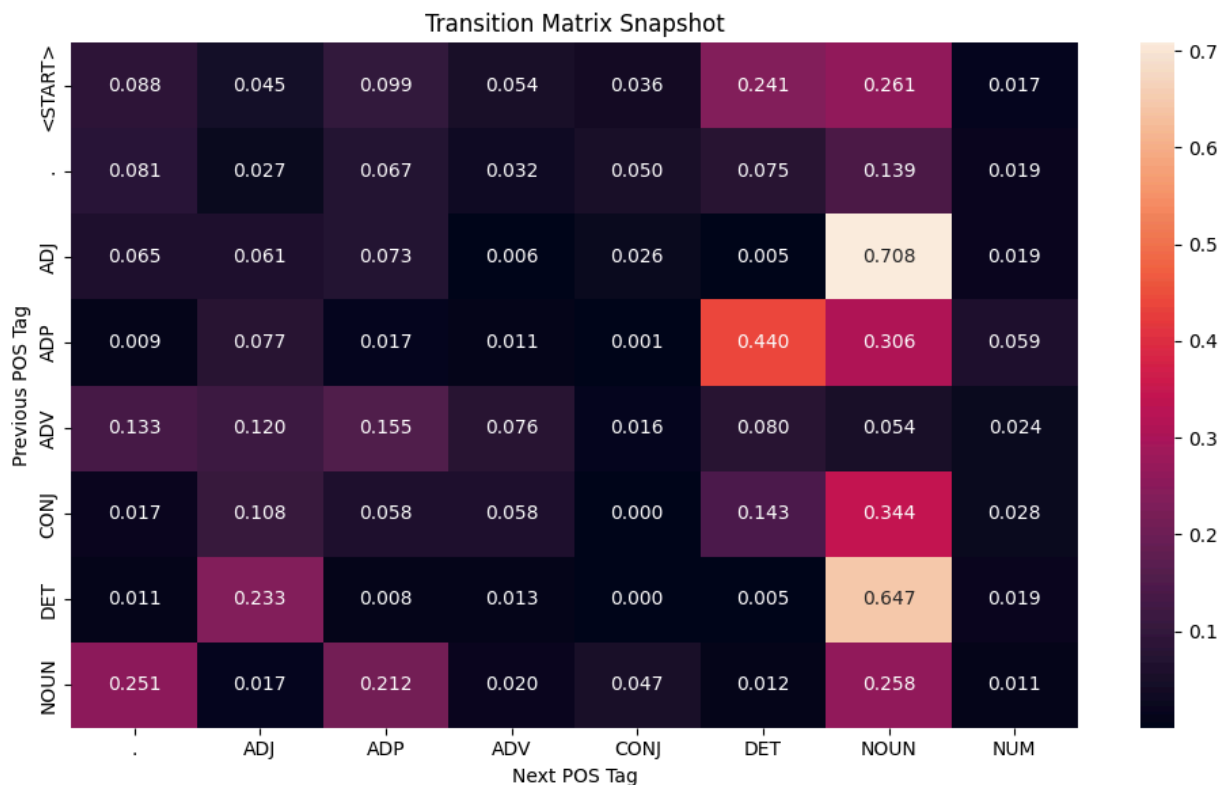
transition_matrix = pd.DataFrame({
    prev_tag: {next_tag: math.exp(transition_log_probs[prev_tag][next_tag]) for next_tag in all_next_tags}
    for prev_tag in all_prev_tags
})
```

```
}).T
transition_matrix.round(4).iloc[:8, :8]
```

```
Out[12]:
```

	.	ADJ	ADP	ADV	CONJ	DET	NOUN	NUM
<START>	0.0876	0.0451	0.0986	0.0544	0.0365	0.2409	0.2608	0.0168
.	0.0815	0.0267	0.0669	0.0325	0.0502	0.0755	0.1393	0.0192
ADJ	0.0650	0.0610	0.0728	0.0055	0.0260	0.0049	0.7083	0.0191
ADP	0.0087	0.0771	0.0169	0.0108	0.0011	0.4398	0.3058	0.0586
ADV	0.1327	0.1202	0.1553	0.0756	0.0161	0.0803	0.0544	0.0244
CONJ	0.0172	0.1077	0.0575	0.0582	0.0004	0.1432	0.3440	0.0278
DET	0.0105	0.2333	0.0082	0.0132	0.0004	0.0053	0.6466	0.0190
NOUN	0.2510	0.0169	0.2120	0.0204	0.0475	0.0122	0.2581	0.0105

```
In [13]: plt.figure(figsize=(10, 6))
sns.heatmap(transition_matrix.loc[all_prev_tags[:8], all_next_tags[:8]], annot=True, fmt=".3f")
plt.title("Transition Matrix Snapshot")
plt.xlabel("Next POS Tag")
plt.ylabel("Previous POS Tag")
plt.tight_layout()
plt.show()
```



2(c) Maximum Likelihood Estimation: Emission Matrix (B)

```
In [14]: emission_counts = defaultdict(Counter)
tag_token_counts = Counter()

for sentence in train_sentences_raw:
    for word, tag in sentence:
        word_norm = normalize_word(word)
        emission_counts[tag][word_norm] += 1
        tag_token_counts[tag] += 1

vocab_with_unk = set(train_vocabulary)
vocab_with_unk.add(UNK_TOKEN)
vocab_size = len(vocab_with_unk)

emission_log_probs = defaultdict(dict)
for tag in states:
    denominator = tag_token_counts[tag] + vocab_size
```

```

for word in vocab_with_unk:
    numerator = emission_counts[tag][word] + 1
    emission_log_probs[tag][word] = math.log(numerator / denominator)

most_common_words = [word for word, count in Counter(normalize_word(w) for w in train_words_original).most_common(10)]
emission_snapshot = pd.DataFrame({
    tag: {word: math.exp(emission_log_probs[tag][word]) for word in most_common_words}
    for tag in states
}).T

emission_snapshot.round(5)

```

Out [14]:

	the	,	.	of	and	to	a	in	for	that
.	0.00004	0.20722	0.16098	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004
ADJ	0.00005	0.00005	0.00005	0.00005	0.00005	0.00005	0.00005	0.00005	0.00005	0.00005
ADP	0.00004	0.00004	0.00004	0.11238	0.00004	0.03534	0.00004	0.07755	0.03801	0.02148
ADV	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00018	0.00036
CONJ	0.00006	0.00006	0.00006	0.00006	0.13816	0.00006	0.00006	0.00006	0.00006	0.00006
DET	0.26067	0.00004	0.00004	0.00004	0.00004	0.00004	0.08656	0.00004	0.00004	0.00616
NOUN	0.00002	0.00002	0.00002	0.00002	0.00002	0.00005	0.00021	0.00002	0.00002	0.00002
NUM	0.00007	0.00007	0.00007	0.00007	0.00007	0.00007	0.00007	0.00007	0.00007	0.00007
PRON	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00006	0.00824
PRT	0.00007	0.00007	0.00007	0.00007	0.00007	0.08097	0.00007	0.00306	0.00007	0.00007
VERB	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004	0.00004
X	0.00008	0.00008	0.00008	0.00008	0.00008	0.00008	0.00023	0.00008	0.00008	0.00008

2(d) OOV Handling Mechanism

```

In [15]: def word_to_model_vocab(word):
    word_norm = normalize_word(word)
    if word_norm in train_vocabulary:
        return word_norm
    return UNK_TOKEN

oov_demo_words = ["The", "AI", "Prashant", "government", "dragon"]
pd.DataFrame({
    "Original Word": oov_demo_words,
    "Model Word": [word_to_model_vocab(w) for w in oov_demo_words]
})

```

Out [15]:

	Original Word	Model Word
0	The	the
1	AI	<UNK>
2	Prashant	<UNK>
3	government	government
4	dragon	<UNK>

2(e) Viterbi Algorithm from Scratch

```

In [16]: def viterbi_decode(words):
    if len(words) == 0:
        return []

    words_model = [word_to_model_vocab(word) for word in words]
    dp = []
    backpointer = []

    first_word = words_model[0]
    first_dp = {}
    first_bp = {}
    for tag in states:
        first_dp[tag] = transition_log_probs[START_TAG][tag] + emission_log_probs[tag][first_word]
        first_bp[tag] = None

```

```

dp.append(first_dp)
backpointer.append(first_bp)

for t in range(1, len(words_model)):
    current_word = words_model[t]
    current_dp = {}
    current_bp = {}
    for current_tag in states:
        best_prev_tag = None
        best_score = -float("inf")
        for prev_tag in states:
            score = (
                dp[t - 1][prev_tag]
                + transition_log_probs[prev_tag][current_tag]
                + emission_log_probs[current_tag][current_word]
            )
            if score > best_score:
                best_score = score
                best_prev_tag = prev_tag
            current_dp[current_tag] = best_score
            current_bp[current_tag] = best_prev_tag
        dp.append(current_dp)
        backpointer.append(current_bp)

best_last_tag = None
best_final_score = -float("inf")
for tag in states:
    final_score = dp[-1][tag] + transition_log_probs[tag][END_TAG]
    if final_score > best_final_score:
        best_final_score = final_score
        best_last_tag = tag

best_tags = [best_last_tag]
for t in range(len(words_model) - 1, 0, -1):
    best_tags.append(backpointer[t][best_tags[-1]])
best_tags.reverse()
return best_tags

sample_words = [word for word, tag in test_sentences_raw[0]]
sample_actual = [tag for word, tag in test_sentences_raw[0]]
sample_predicted = viterbi_decode(sample_words)

pd.DataFrame({"Word": sample_words, "Actual POS": sample_actual, "Predicted POS": sample_predicted}).he

```

Out[16]:

	Word	Actual POS	Predicted POS
0	Thirty-three	NUM	.

Part 3: Inferences

3(a) Sequence Inference: 2 Training Sentences and 2 Testing Sentences

```

In [17]: def make_prediction_table(sentence, source_name, sentence_number):
    words = [word for word, tag in sentence]
    actual = [tag for word, tag in sentence]
    predicted = viterbi_decode(words)
    return pd.DataFrame({
        "Source": source_name,
        "Sentence No.": sentence_number,
        "Word": words,
        "Actual POS": actual,
        "Predicted POS": predicted,
        "Correct": [a == p for a, p in zip(actual, predicted)]
    })

inference_tables = [
    make_prediction_table(train_sentences_raw[0], "Training: news", 1),
    make_prediction_table(train_sentences_raw[1], "Training: news", 2),
    make_prediction_table(test_sentences_raw[0], "Testing: fiction", 1),
    make_prediction_table(test_sentences_raw[1], "Testing: fiction", 2)
]
inference_output = pd.concat(inference_tables, ignore_index=True)
inference_output

```


Out [17]:

	Source	Sentence No.	Word	Actual POS	Predicted POS	Correct
0	Training: news	1	The	DET	DET	True
1	Training: news	1	Fulton	NOUN	NOUN	True
2	Training: news	1	County	NOUN	NOUN	True
3	Training: news	1	Grand	ADJ	ADJ	True
4	Training: news	1	Jury	NOUN	NOUN	True
...	...	...	...	...	...	...
72	Testing: fiction	2	go	VERB	VERB	True
73	Testing: fiction	2	back	ADV	ADV	True
74	Testing: fiction	2	to	ADP	ADP	True
75	Testing: fiction	2	school	NOUN	NOUN	True
76	Testing: fiction	2	.	.	.	True

77 rows × 6 columns

```

In [18]: inference_summary_rows = []
for source, sent_no, sentence in [
    ("Training: news", 1, train_sentences_raw[0]),
    ("Training: news", 2, train_sentences_raw[1]),
    ("Testing: fiction", 1, test_sentences_raw[0]),
    ("Testing: fiction", 2, test_sentences_raw[1]),
]:
    words = [word for word, tag in sentence]
    actual = [tag for word, tag in sentence]
    predicted = viterbi_decode(words)
    inference_summary_rows.append({
        "Source": source,
        "Sentence No.": sent_no,
        "Sentence": " ".join(words),
        "Actual Tags": " ".join(actual),
        "Predicted Tags": " ".join(predicted),
        "Sentence Accuracy": round(accuracy_score(actual, predicted), 4)
    })
pd.DataFrame(inference_summary_rows)

```

Out [18]:

	Source	Sentence No.	Sentence	Actual Tags	Predicted Tags	Sentence Accuracy
0	Training: news	1	The Fulton County Grand Jury said Friday an in...	DET NOUN NOUN ADJ NOUN VERB NOUN DET NOUN ADP ...	DET NOUN NOUN ADJ NOUN VERB ADP DET NOUN ADP D...	0.9200
1	Training: news	2	The jury further said in term-end presentments...	DET NOUN ADV VERB ADP NOUN NOUN ADP DET NOUN A...	DET NOUN ADV VERB ADP DET NOUN ADP DET NOUN NO...	0.9535
2	Testing: fiction	1	Thirty-three	NUM	.	0.0000
3	Testing: fiction	2	Scotty did not go back to school .	NOUN VERB ADV VERB ADV ADP NOUN .	PRON VERB ADV VERB ADV ADP NOUN .	0.8750

3(b) Three Misclassified Sentences and Analysis

```

In [19]: def sentence_error_reason(words, actual, predicted):
    reasons = []
    oov_in_sentence = [w for w in words if normalize_word(w) not in train_vocabulary]
    if oov_in_sentence:
        reasons.append("OOV words: " + " ".join(oov_in_sentence[:5]))
    mismatches = [(w, a, p) for w, a, p in zip(words, actual, predicted) if a != p]
    if mismatches:
        examples = "; ".join([f"{w}: actual={a}, predicted={p}" for w, a, p in mismatches[:5]])
        reasons.append("Tag mismatch: " + examples)
    reasons.append("Training category is news while testing category is fiction")
    return " | ".join(reasons)

misclassified_sentence_rows = []
misclassified_token_tables = []

for idx, sentence in enumerate(test_sentences_raw):

```

```

words = [word for word, tag in sentence]
actual = [tag for word, tag in sentence]
predicted = viterbi_decode(words)
if any(a != p for a, p in zip(actual, predicted)):
    example_no = len(misclassified_sentence_rows) + 1
    misclassified_sentence_rows.append({
        "Example No.": example_no,
        "Test Sentence Index": idx,
        "Sentence": " ".join(words),
        "Sentence Accuracy": round(accuracy_score(actual, predicted), 4),
        "Analysis": sentence_error_reason(words, actual, predicted)
    })
    misclassified_token_tables.append(pd.DataFrame({
        "Example No.": example_no,
        "Word": words,
        "Actual POS": actual,
        "Predicted POS": predicted,
        "Correct": [a == p for a, p in zip(actual, predicted)]
    })))
if len(misclassified_sentence_rows) == 3:
    break

misclassified_summary = pd.DataFrame(misclassified_sentence_rows)
misclassified_summary

```

Out [19]:

	Example No.	Test Sentence Index	Sentence	Sentence Accuracy	Analysis
0	1	0	Thirty-three	0.0000	OOV words: Thirty-three Tag mismatch: Thirty...
1	2	1	Scotty did not go back to school .	0.8750	OOV words: Scotty Tag mismatch: Scotty: actu...
2	3	2	His parents talked seriously and lengthily to ...	0.9425	OOV words: lengthily, specialist, McKinley, di...

In [20]: `misclassified_details = pd.concat(misclassified_token_tables, ignore_index=True)`
`misclassified_details`

Out [20]:

	Example No.	Word	Actual POS	Predicted POS	Correct
0	1	Thirty-three	NUM	.	False
1	2	Scotty	NOUN	PRON	False
2	2	did	VERB	VERB	True
3	2	not	ADV	ADV	True
4	2	go	VERB	VERB	True
...	...	...	...	...	...
91	3	exciting	ADJ	ADJ	True
92	3	or	CONJ	CONJ	True
93	3	too	ADV	ADV	True
94	3	debilitating	ADJ	VERB	False
95	3	.	.	.	True

96 rows × 5 columns

3(c) Full Test Set Evaluation

In [21]:

```

y_true = []
y_pred = []

for sentence in test_sentences_raw:
    words = [word for word, tag in sentence]
    actual = [tag for word, tag in sentence]
    predicted = viterbi_decode(words)
    y_true.extend(actual)
    y_pred.extend(predicted)

hmm_accuracy = accuracy_score(y_true, y_pred)

pd.DataFrame({

```

```
"Metric": ["Total Test Tokens", "Correct Predictions", "Overall Token-Level Accuracy"],
"Value": [len(y_true), sum(a == p for a, p in zip(y_true, y_pred)), round(hmm_accuracy, 4)]
})
```

Out [21]:

	Metric	Value
0	Total Test Tokens	68488.0000
1	Correct Predictions	60085.0000
2	Overall Token-Level Accuracy	0.8773

In [22]:

```
print("Classification Report: HMM POS Tagger")
print(classification_report(y_true, y_pred, labels=states, zero_division=0))
```

```
Classification Report: HMM POS Tagger
              precision    recall  f1-score   support

     .           0.98         1.00         0.99         10170
    ADJ           0.70         0.65         0.67          3684
    ADP           0.84         0.97         0.90          7198
    ADV           0.92         0.70         0.79          3716
   CONJ           1.00         0.99         0.99          2261
    DET           0.86         0.98         0.91          8018
   NOUN           0.83         0.84         0.83         13509
    NUM           0.96         0.76         0.85           446
   PRON           0.87         0.92         0.89          4799
    PRT           0.88         0.71         0.79          2305
   VERB           0.91         0.83         0.87         12299
     X           1.00         0.11         0.20           83

 accuracy                   0.88         68488
 macro avg           0.89         0.79         0.81         68488
 weighted avg           0.88         0.88         0.88         68488
```

In [23]:

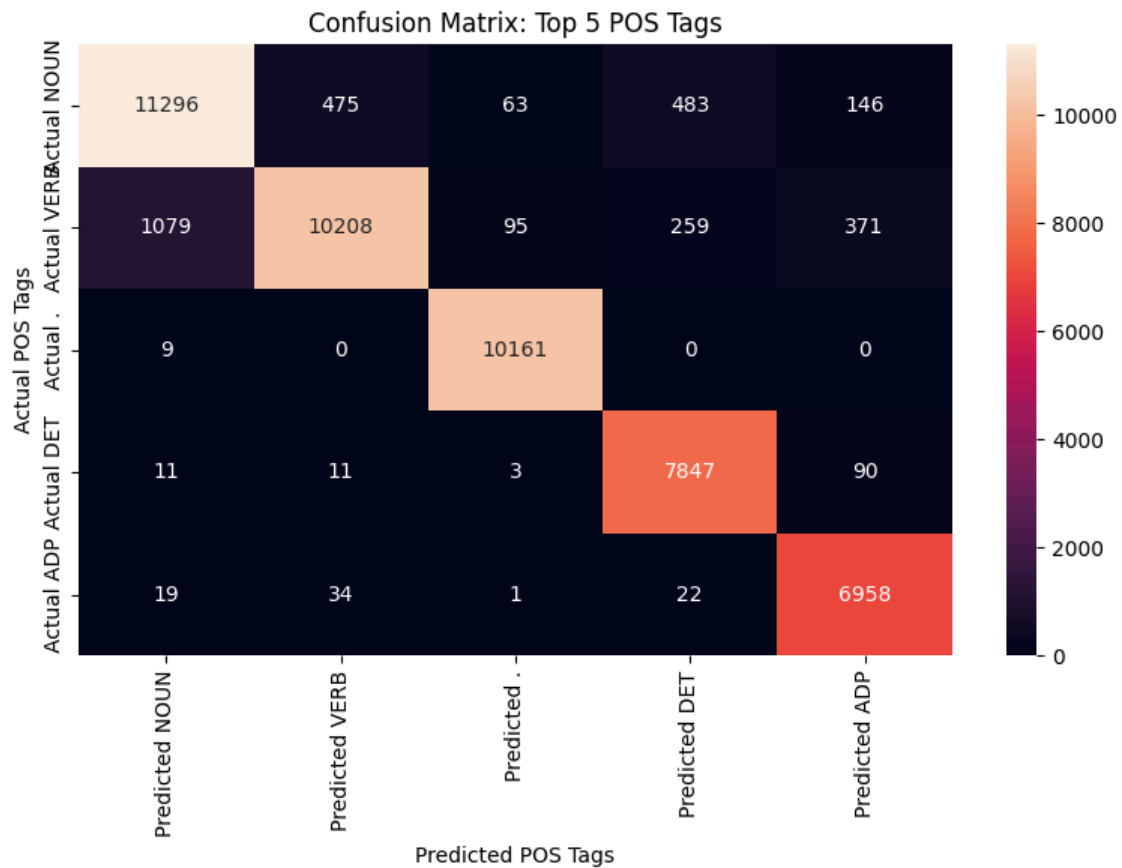
```
top_5_tags = [tag for tag, count in Counter(y_true).most_common(5)]
cm_top5 = confusion_matrix(y_true, y_pred, labels=top_5_tags)
cm_top5_df = pd.DataFrame(
    cm_top5,
    index=[f"Actual {tag}" for tag in top_5_tags],
    columns=[f"Predicted {tag}" for tag in top_5_tags]
)
cm_top5_df
```

Out [23]:

	Predicted NOUN	Predicted VERB	Predicted .	Predicted DET	Predicted ADP
Actual NOUN	11296	475	63	483	146
Actual VERB	1079	10208	95	259	371
Actual .	9	0	10161	0	0
Actual DET	11	11	3	7847	90
Actual ADP	19	34	1	22	6958

In [24]:

```
plt.figure(figsize=(8, 6))
sns.heatmap(cm_top5_df, annot=True, fmt="d")
plt.title("Confusion Matrix: Top 5 POS Tags")
plt.xlabel("Predicted POS Tags")
plt.ylabel("Actual POS Tags")
plt.tight_layout()
plt.show()
```



Part D: Baseline Comparison

D(a) Pre-trained POS Tagger on the Same Test Set

```
In [25]: def nltk_baseline_tag(words):
    return [tag for word, tag in nltk.pos_tag(words, tagset="universal")]

baseline_true = []
baseline_pred = []

for sentence in test_sentences_raw:
    words = [word for word, tag in sentence]
    actual = [tag for word, tag in sentence]
    predicted = nltk_baseline_tag(words)
    baseline_true.extend(actual)
    baseline_pred.extend(predicted)

baseline_accuracy = accuracy_score(baseline_true, baseline_pred)

pd.DataFrame({
    "Model": ["Custom HMM + Viterbi", "NLTK PerceptronTagger"],
    "Test Category": ["fiction", "fiction"],
    "Token-Level Accuracy": [round(hmm_accuracy, 4), round(baseline_accuracy, 4)]
})
```

```
Out[25]:
```

	Model	Test Category	Token-Level Accuracy
0	Custom HMM + Viterbi	fiction	0.8773
1	NLTK PerceptronTagger	fiction	0.9069

```
In [26]: baseline_sample_rows = []
for sent_no, sentence in [(1, test_sentences_raw[0]), (2, test_sentences_raw[1])]:
    words = [word for word, tag in sentence]
    actual = [tag for word, tag in sentence]
    hmm_pred = viterbi_decode(words)
    baseline_pred = nltk_baseline_tag(words)
    for word, actual_tag, hmm_tag, baseline_tag in zip(words, actual, hmm_pred, baseline_pred):
        baseline_sample_rows.append({
            "Test Sentence No.": sent_no,
            "Word": word,
```

```

        "Actual POS": actual_tag,
        "HMM Predicted POS": hmm_tag,
        "Baseline Predicted POS": baseline_tag
    })

pd.DataFrame(baseline_sample_rows)

```

Out[26]:

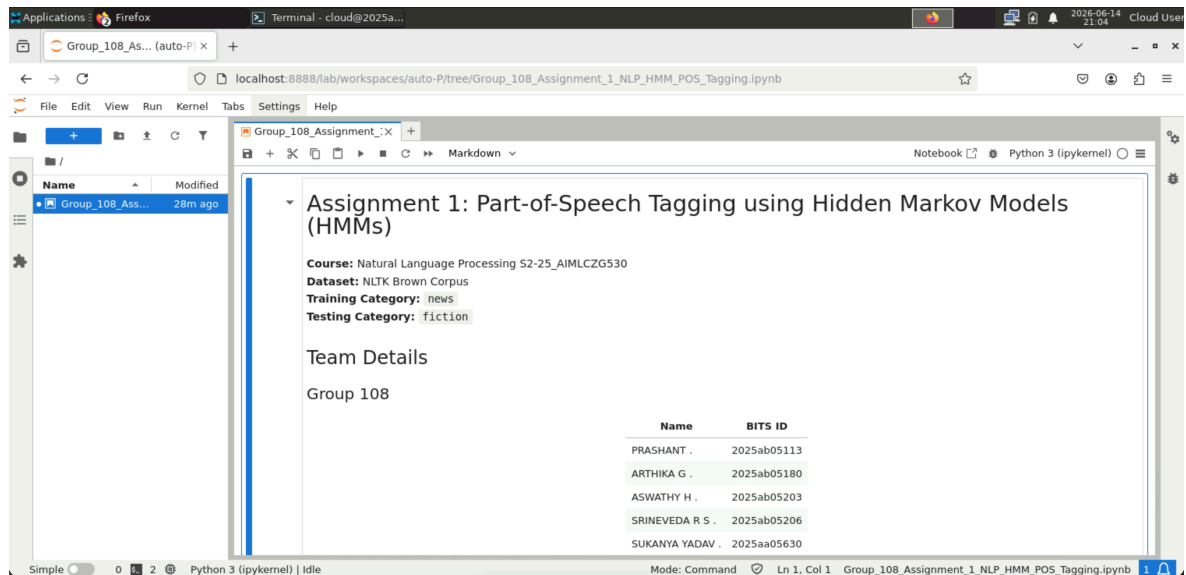
	Test Sentence No.	Word	Actual POS	HMM Predicted POS	Baseline Predicted POS
0	1	Thirty-three	NUM	.	ADJ
1	2	Scotty	NOUN	PRON	ADV
2	2	did	VERB	VERB	VERB
3	2	not	ADV	ADV	ADV
4	2	go	VERB	VERB	VERB
5	2	back	ADV	ADV	ADV
6	2	to	ADP	ADP	PRT
7	2	school	NOUN	NOUN	NOUN
8	2	.	.	.	.

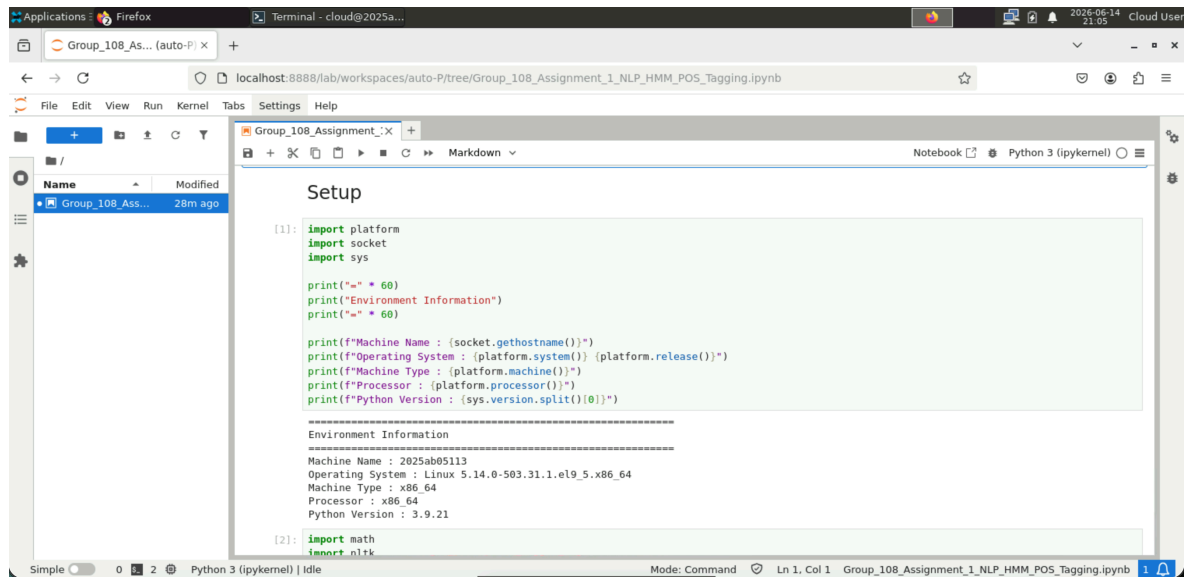
D(b) Discussion

The custom HMM model was trained only on the Brown Corpus "news" category and therefore learned transition and emission probabilities specific to that domain. When evaluated on the "fiction" category, the model encountered different vocabulary and sentence structures, which resulted in lower accuracy.

Since the implemented HMM relies only on the previous POS tag (bigram assumption), it cannot effectively capture longer contextual dependencies. In contrast, the NLTK Perceptron Tagger uses richer contextual features and supervised learning, enabling it to perform better on unseen words and ambiguous grammatical structures.

Screenshots





The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The notebook is titled "Setup". The code in the first cell prints environment information, including machine name, operating system, machine type, processor, and Python version. The output of the code is displayed below the code cell.

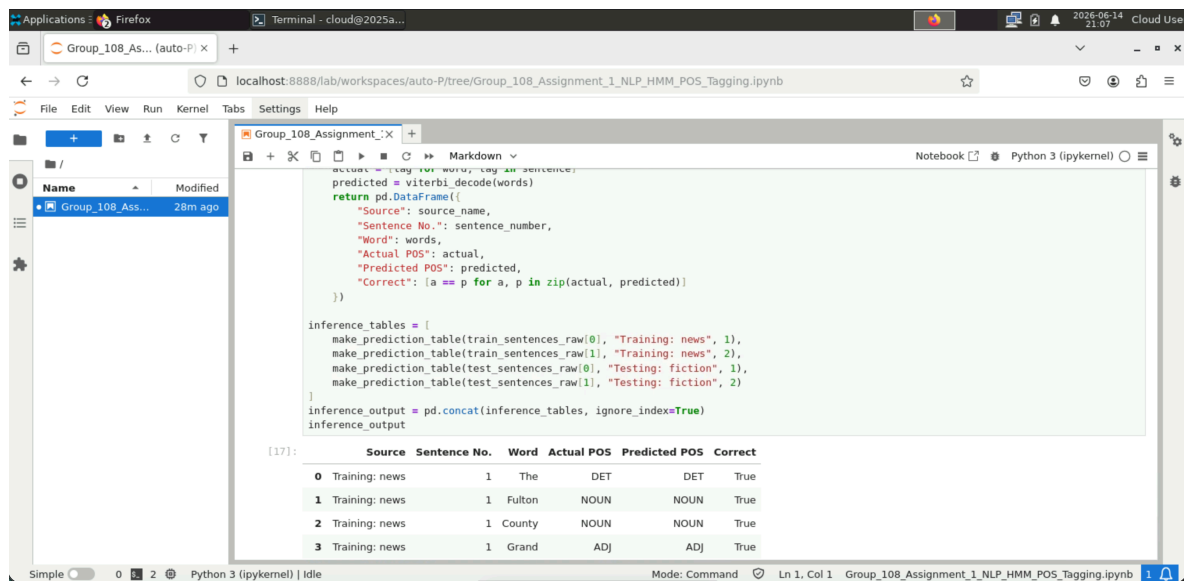
```
[1]: import platform
import socket
import sys

print("-" * 60)
print("Environment Information")
print("-" * 60)

print(f"Machine Name : {socket.gethostname()}")
print(f"Operating System : {platform.system()} {platform.release()}")
print(f"Machine Type : {platform.machine()}")
print(f"Processor : {platform.processor()}")
print(f"Python Version : {sys.version.split()[0]}")

Environment Information
-----
Machine Name : 2025ab05113
Operating System : Linux 5.14.0-503.31.1.el9_5.x86_64
Machine Type : x86_64
Processor : x86_64
Python Version : 3.9.21

[2]: import math
import nltk
```

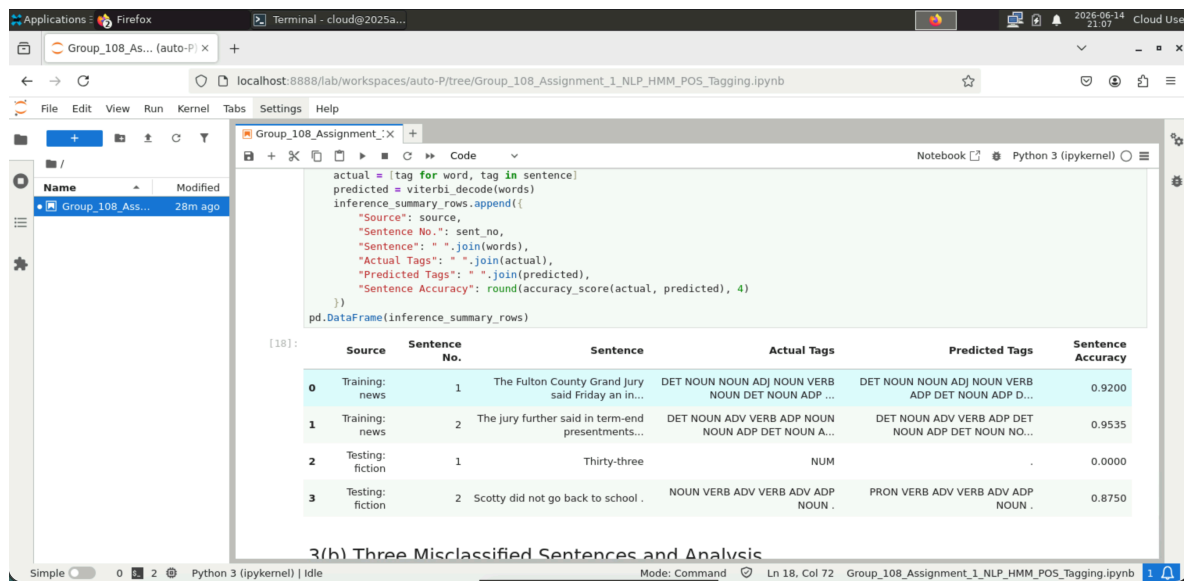


The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The notebook is titled "Inference". The code in the first cell defines a function to make predictions and a function to make inference tables. The output of the code is displayed below the code cell, showing a table of inference results.

```
def make_prediction(words):
    predicted = viterbi_decode(words)
    return pd.DataFrame({
        "Source": source_name,
        "Sentence No.": sentence_number,
        "Word": words,
        "Actual POS": actual,
        "Predicted POS": predicted,
        "Correct": [a == p for a, p in zip(actual, predicted)]
    })

inference_tables = [
    make_prediction_table(train_sentences_raw[0], "Training: news", 1),
    make_prediction_table(train_sentences_raw[1], "Training: news", 2),
    make_prediction_table(test_sentences_raw[0], "Testing: fiction", 1),
    make_prediction_table(test_sentences_raw[1], "Testing: fiction", 2)
]
inference_output = pd.concat(inference_tables, ignore_index=True)
inference_output
```

	Source	Sentence No.	Word	Actual POS	Predicted POS	Correct
0	Training: news	1	The	DET	DET	True
1	Training: news	1	Fulton	NOUN	NOUN	True
2	Training: news	1	County	NOUN	NOUN	True
3	Training: news	1	Grand	ADJ	ADJ	True



The screenshot shows a Jupyter Notebook interface with a file explorer on the left and a code editor on the right. The notebook is titled "Inference". The code in the first cell defines a function to make predictions and a function to make inference tables. The output of the code is displayed below the code cell, showing a table of inference results.

```
actual = [tag for word, tag in sentence]
predicted = viterbi_decode(words)
inference_summary_rows.append({
    "Source": source,
    "Sentence No.": sent_no,
    "Sentence": " ".join(words),
    "Actual Tags": " ".join(actual),
    "Predicted Tags": " ".join(predicted),
    "Sentence Accuracy": round(accuracy_score(actual, predicted), 4)
})
pd.DataFrame(inference_summary_rows)
```

	Source	Sentence No.	Sentence	Actual Tags	Predicted Tags	Sentence Accuracy
0	Training: news	1	The Fulton County Grand Jury said Friday an in...	DET NOUN NOUN ADJ NOUN VERB NOUN DET NOUN ADP ...	DET NOUN NOUN ADJ NOUN VERB ADP DET NOUN ADP D...	0.9200
1	Training: news	2	The jury further said in term-end presentments...	DET NOUN ADV VERB ADP NOUN NOUN DET NOUN A...	DET NOUN ADV VERB ADP DET NOUN ADP DET NOUN NO...	0.9535
2	Testing: fiction	1	Thirty-three	NUM	.	0.0000
3	Testing: fiction	2	Scotty did not go back to school .	NOUN VERB ADV VERB ADV ADP NOUN .	PRON VERB ADV VERB ADV ADP NOUN .	0.8750

3(b) Three Misclassified Sentences and Analysis

In []: